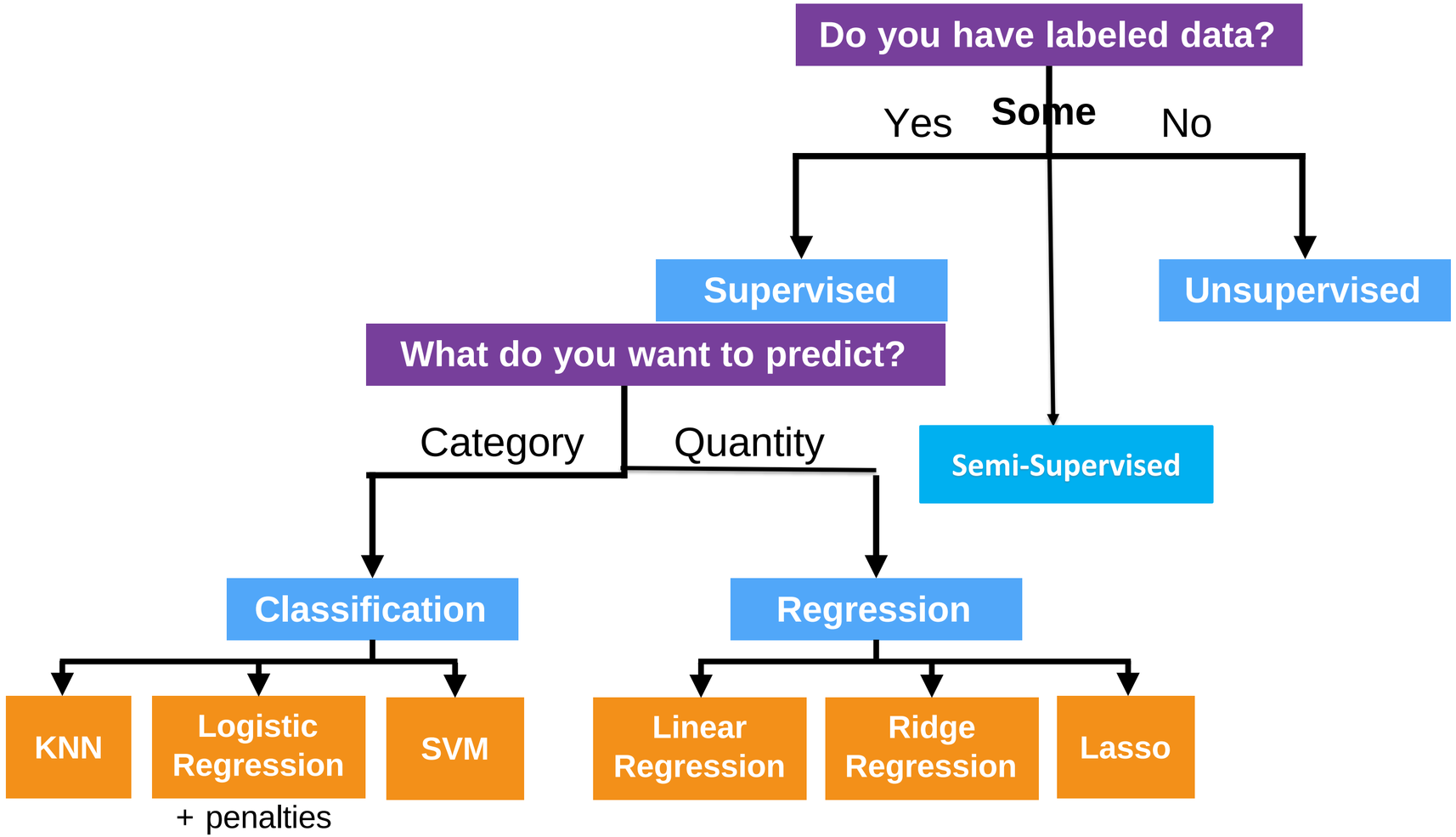
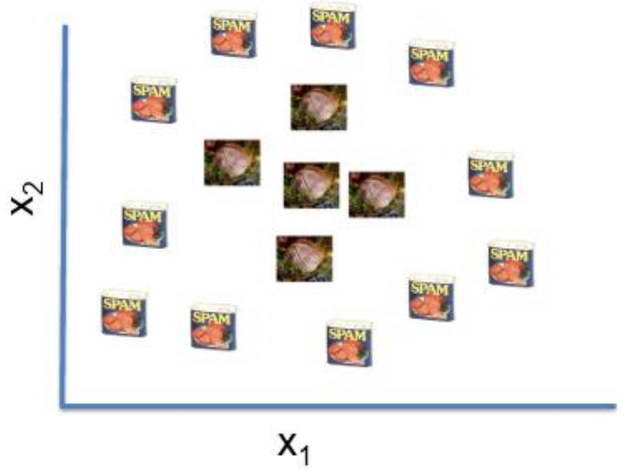
Support Vector Machine (SVM)

Machine Learning Methods



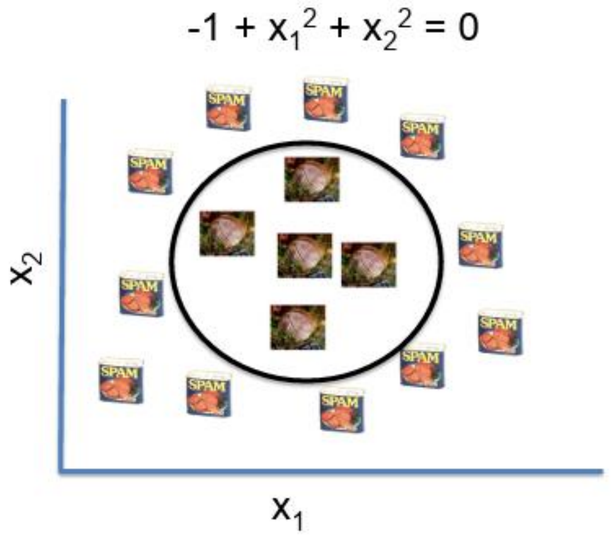
How to handle a non-linear decision boundary



- This data is not linearly separable in the original feature space
- Use non-linear basis functions to add more features, hopefully it becomes linearly separable in the "augmented" space.

Adding Polynomial features

- New feature vector is $\mathbf{x} = [1, x_1, x_2, x_1^2, x_2^2]$
- $\Pr(y = 1|\mathbf{x}) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2)$
- If $\mathbf{w} = [-1, 0, 0, 1, 1]$, the boundary is $-1 + x_1^2 + x_2^2 = 0$
  - If $-1 + x_1^2 + x_2^2 \geq 0$ declare spam
  - If $-1 + x_1^2 + x_2^2 < 0$ declare ham

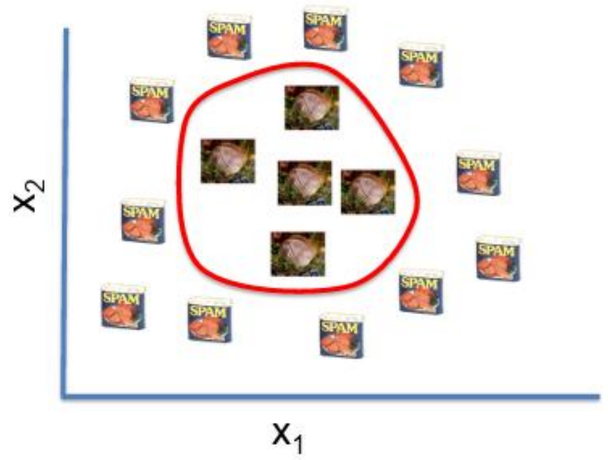


Add Regularisation for balance

- Add regularization term to be cross entropy loss function

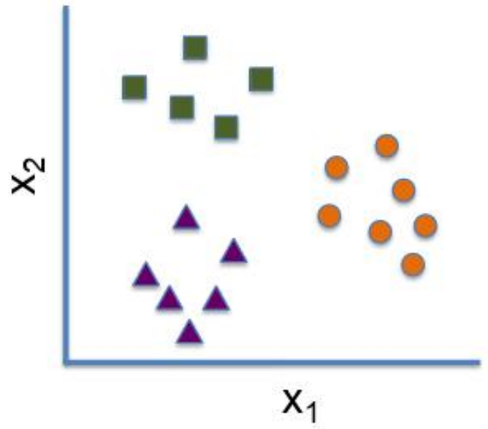
$$\mathcal{E}(\boldsymbol{w}) = -\sum_n \{y_n \log \sigma(\boldsymbol{w}^\top \boldsymbol{x}_n) + (1-y_n) \log[1-\sigma(\boldsymbol{w}^\top \boldsymbol{x}_n)]\} + \underbrace{\frac{1}{2}\lambda \|w\|_2^2}_{\text{regularization}}$$

- Perform gradient descent on this regularized function
- Often, we do NOT regularize the bias term w_0



Multiclass Logistic Regression

- One-versus-all
- One-versus-one
- Multinomial regression

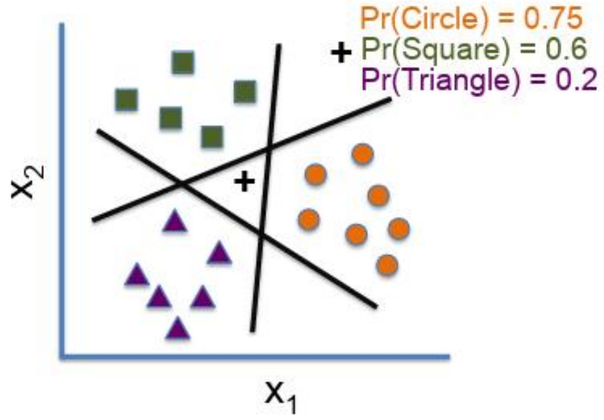


Combining Decision Boundaries

How to combine these linear decision boundaries?

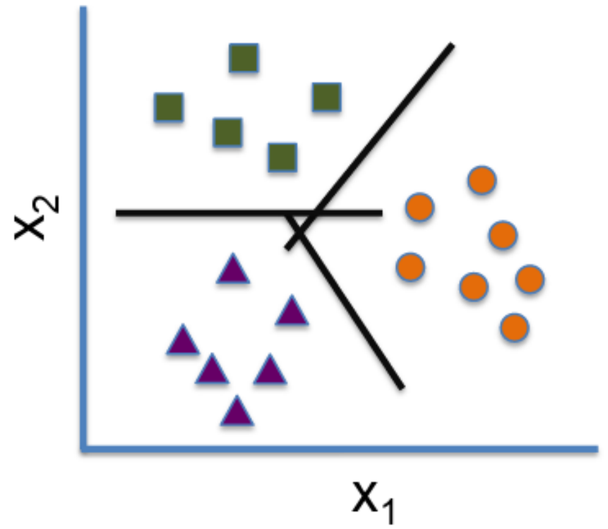
- Use the confidence estimates $\Pr(y = C_1|\mathbf{x}) = \sigma(\mathbf{w}_1^\top \mathbf{x})$, $\ldots \Pr(y = C_K|\mathbf{x}) = \sigma(\mathbf{w}_K^\top \mathbf{x})$
- Declare class C_k^* that maximizes

$$k^* = \arg \max_{k=1,\ldots,K} \Pr(y = C_k|\mathbf{x}) = \sigma(\mathbf{w}_k^\top \mathbf{x})$$



One versus One

- How many binary classifiers for K classes? $K(K-1)/2$
- How to combine their outputs?



Multinomial Logistic Regression

- Model: For each class C_k, we have a parameter vector $\boldsymbol{w}_k$ and model the posterior probability as:

$$p(C_k|\boldsymbol{x}) = \frac{e^{\boldsymbol{w}_k^\top \boldsymbol{x}}}{\sum_{k'} e^{\boldsymbol{w}_{k'}^\top \boldsymbol{x}}} \quad \leftarrow \quad \text{This is called } \textit{softmax} \text{ function}$$

- Decision boundary / testing: Assign $\boldsymbol{x}$ with the label that is the maximum of posterior:

$$\arg\max_k P(C_k|\boldsymbol{x}) \rightarrow \arg\max_k \boldsymbol{w}_k^\top \boldsymbol{x}.$$

Support Vector Machines

Support vector machine (SVM) is a supervised method for binary classification (two class). It is a generalization of 1 and 2 below.

1. **Maximal margin classifier:** only applicable to linearly separable data.
2. **Support vector classifier:** can be applied to data that is not linearly separable. Decision boundary still linear.
3. **Support vector machine:** non-linear decision boundary.

Important to Note

1. Binary SVM can be paired with the previously mentioned method to classify multiple classes.

2. The support vector machine is a Generalization of the support vector classifier that classifies the observations using nonlinear boundaries by expanding the feature space with the help of kernels (James et al. 2013).

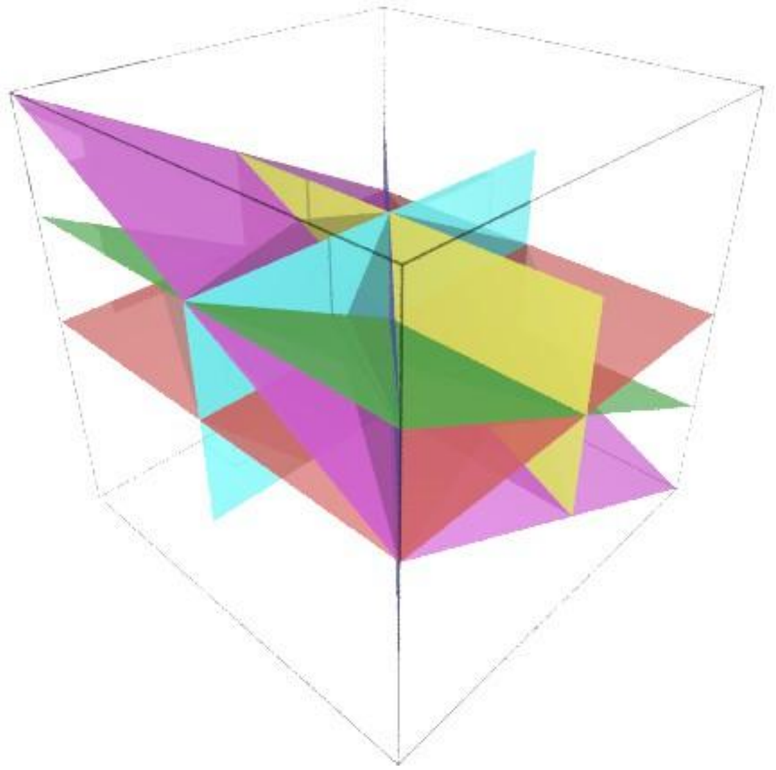
Hyperplanes

What is a hyperplane?

In p-dimensional space, a hyperplane is a $(p\text{-}1)$-dimensional affine subspace.

In 2D, a hyperplane is a flat 1D subspace, aka a line.

In 3D, a hyperplane is a flat 2D subspace, aka a plane.



Mathematical Definition

A 2D hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

By “define”, we mean that any $X = (X_1, X_2)$ for which the above equation holds is a point on the hyperplane.

Note that the above is the equation of a line, aka a hyperplane in 2D.

Mathematical Definition

In p dimensions, a hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p = 0$$

Similarly, any $X = (X_1, X_2, \ldots, X_p)$ for which the above equation holds is a point on the hyperplane.

Separating Hyperplane

Instead of a point on the hyperplane, consider $\boldsymbol{X}$ for which

$$\beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p > 0$$

This point lies on one side of the hyperplane. An $\boldsymbol{X}$ for which

$$\beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p < 0$$

lies on the other side of the hyperplane.

We can think of the hyperplane as dividing the p-dimensional space into two halves.

Separating Hyperplane

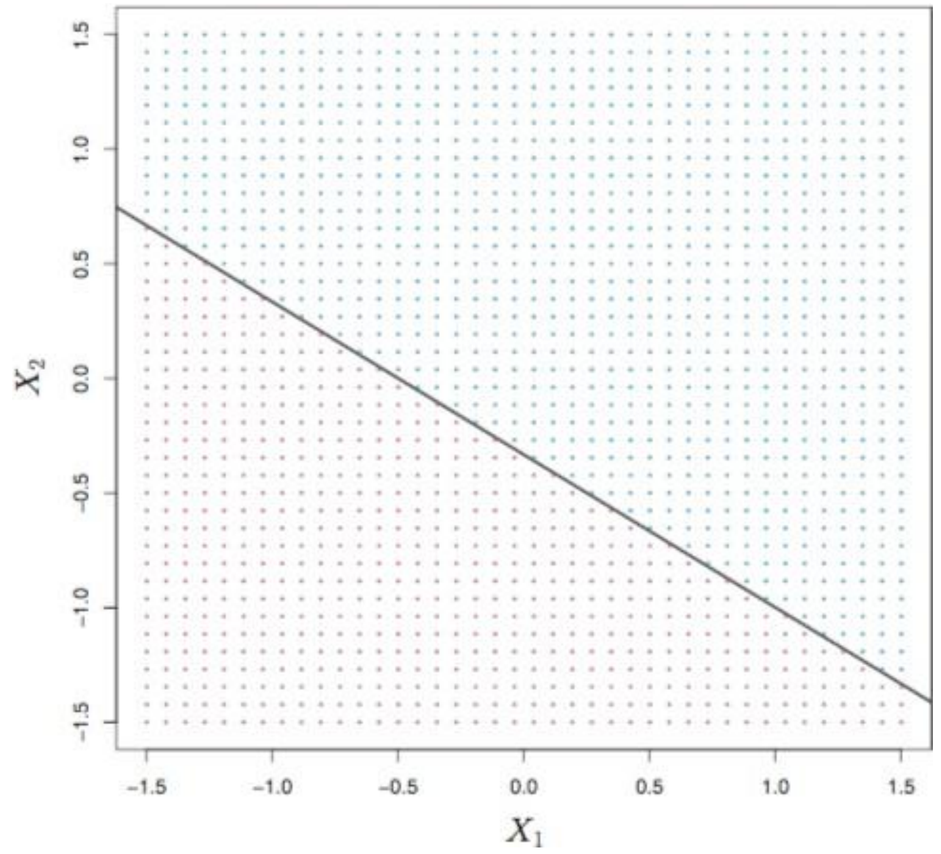


FIGURE 9.1, ISL (8th printing 2017)

This hyperplane in 2 dimensions is the line $1+2X_1+3X_2 = 0$.

The blue region is the set of points for which $1+2X_1+3X_2 > 0$.

The purple region is the set of points for which $1+2X_1+3X_2 < 0$.

Separating Hyperplane Classifier

Idea: Use a separating hyperplane for binary classification.

Key assumption: Classes can be separated by a linear decision boundary.

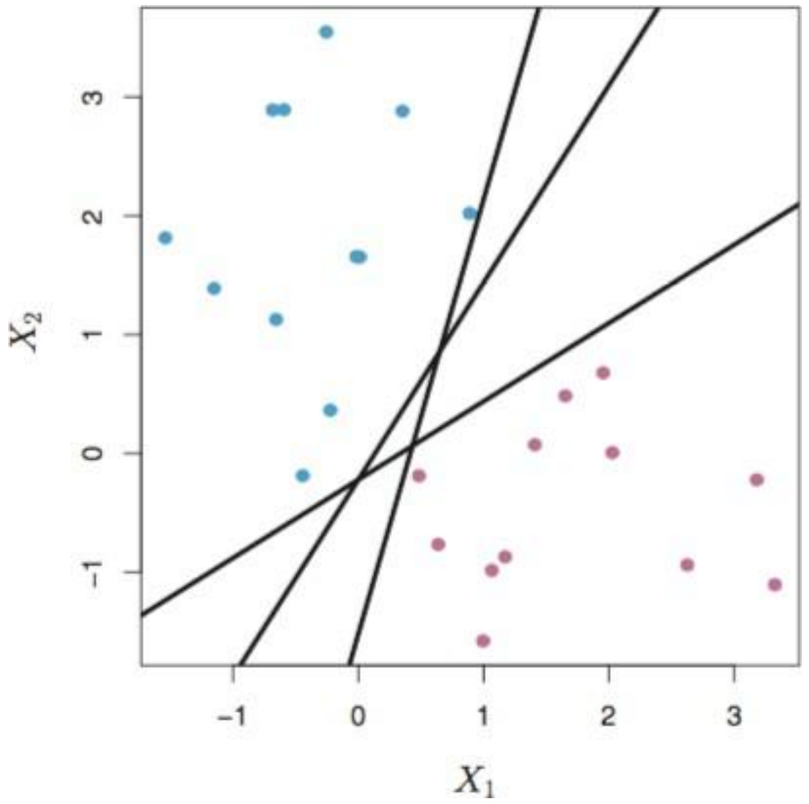
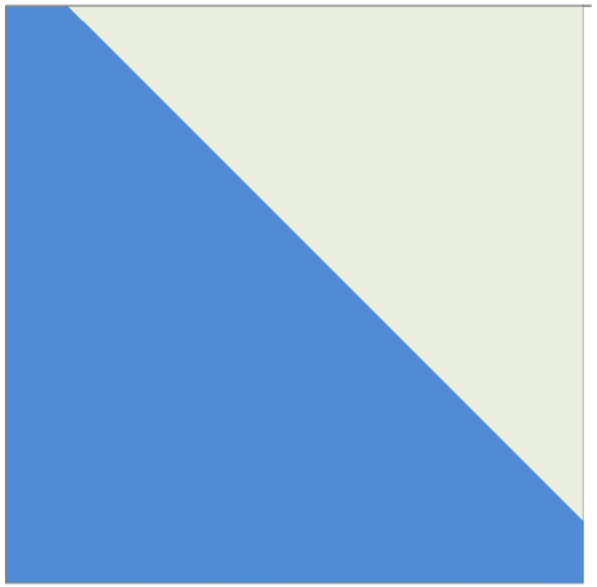
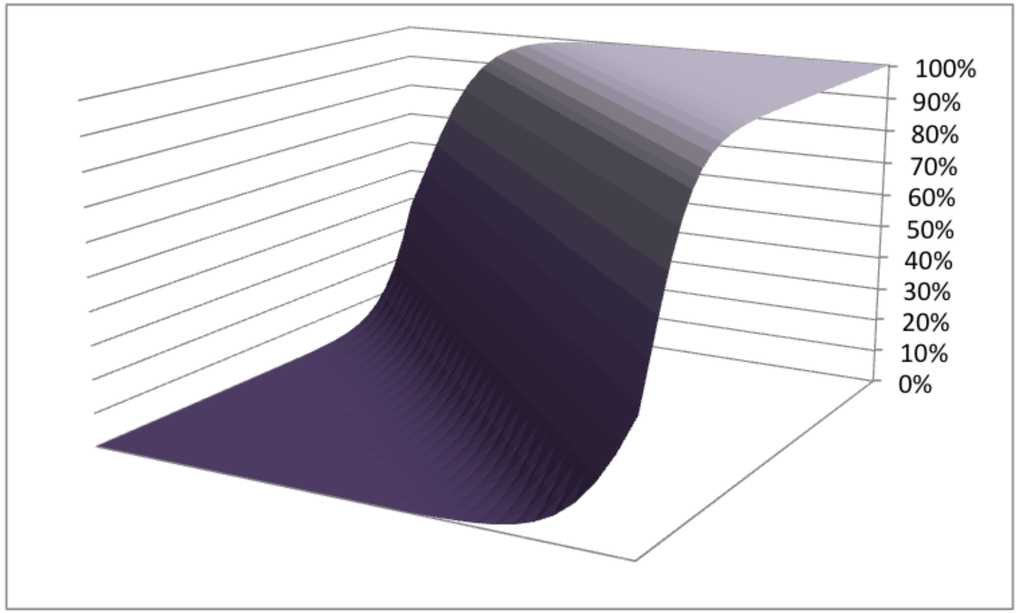


FIGURE 9.2, ISL (8th printing 2017)

Separating Hyperplane Classifier

Aside: Logistic regression effectively finds a separating hyperplane.





Maximal margin classifiers and SVMs do this differently.

Separating Hyperplane Classifier

To classify new data points:

Assign class by location of new data point with respect to the hyperplane.

$$\hat{y} = \text{sign}(\beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p)$$

The farther away a point is from the separating hyperplane, the more confident we are about its class assignment.

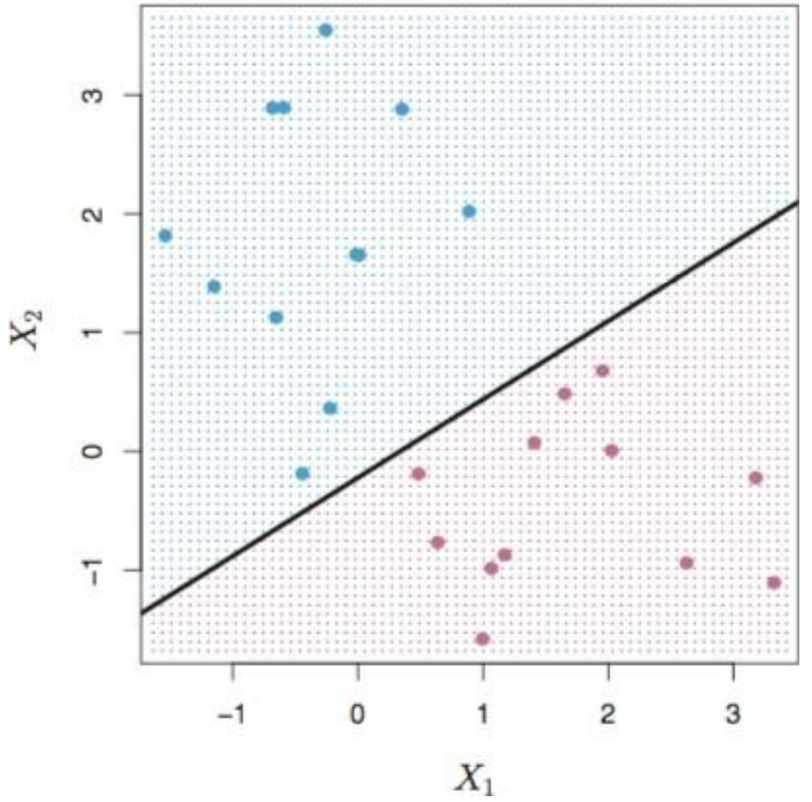


FIGURE 9.2, ISL (8th printing 2017)

Separating Hyperplane Classifier

Notice that for a linearly separable dataset, there are many possible separating hyperplanes that divide the dataset into two classes (in fact, an infinite number).

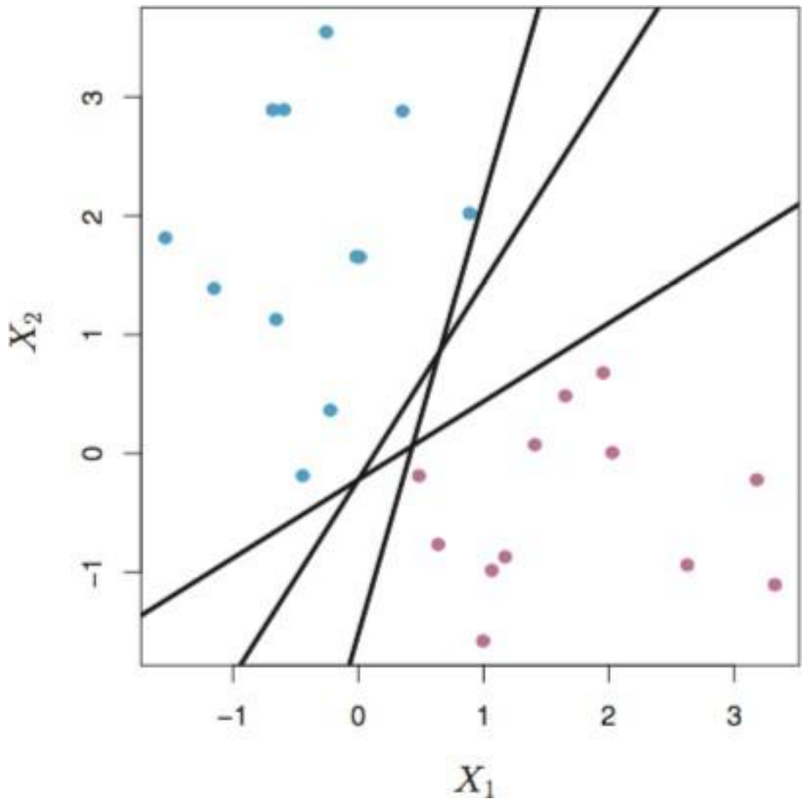
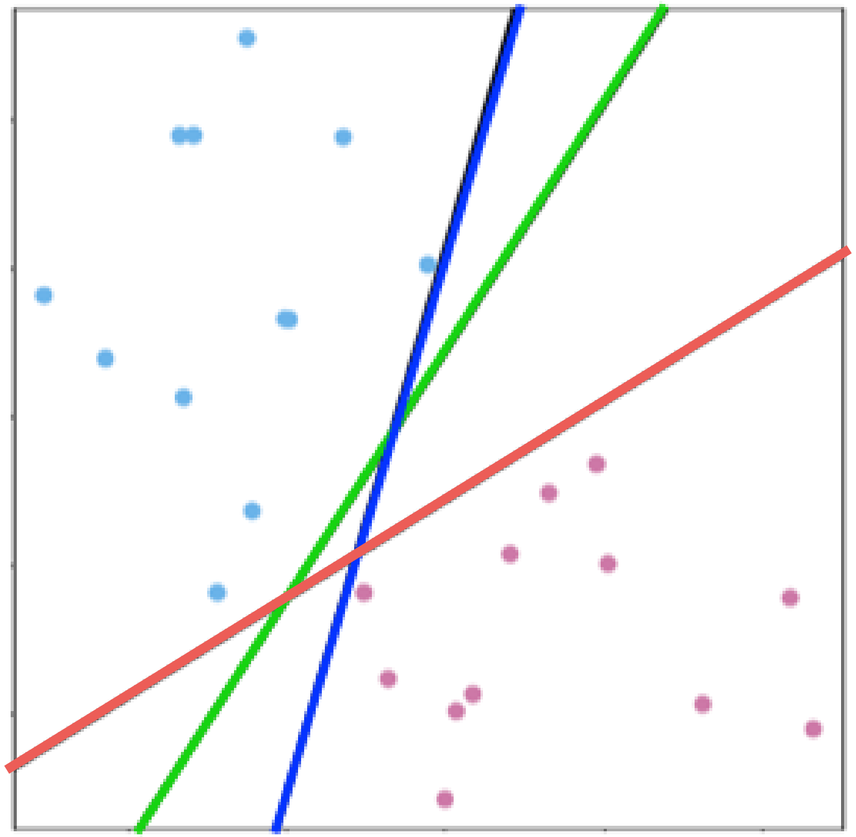


FIGURE 9.2, ISL (8th printing 2017)

Maximal Margin Classifier

Which decision boundary?



Maximal Margin Hyperplane

Which of the infinite separating hyperplanes should we choose?

A natural choice is the **maximal margin hyperplane**, the separating hyperplane that is farthest from the training samples.

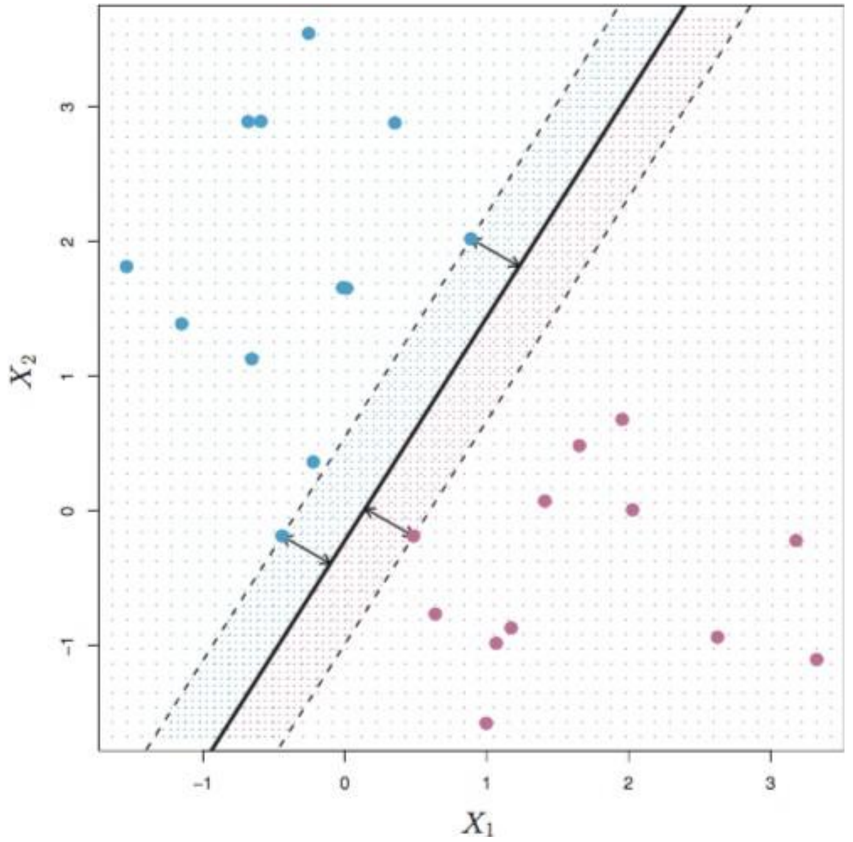


FIGURE 9.3, ISL (8th printing 2017)

Maximal Margin Hyperplane

Margin: smallest distance between any training observation and the hyperplane.

Support vectors: training observations whose distance to the hyperplane is equal to the margin

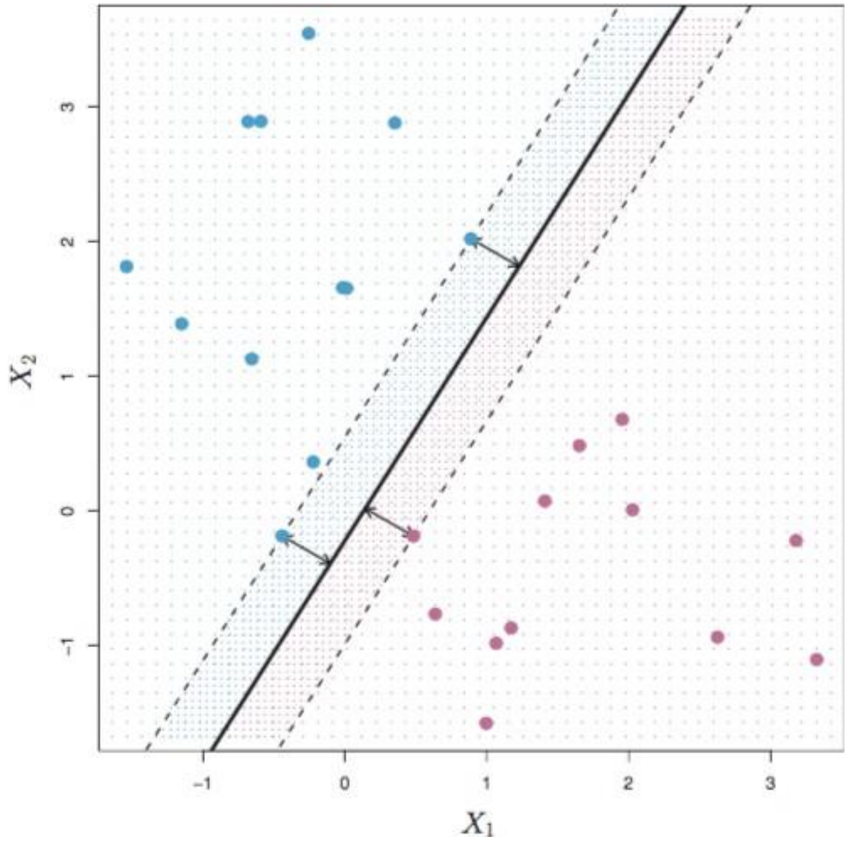


FIGURE 9.3, ISL (8th printing 2017)

Why is it called a support vector?

- **“Support”:** maximal margin hyperplane only depends on these observations.
- **“Vector”:** points are vectors in *p*-dimensional space.
- If support vectors are perturbed, then MM hyperplane will change.
- If other training observations perturbed (provided not perturbed within margin distance of hyperplane), then MM hyperplane not affected.

Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$ solve:

$$\max_{\beta_0,\ldots,\beta_p} M$$

maximize the margin, M

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

constraint necessary for well-defined optimization problem

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

all training points must be at least distance M from hyperplane

Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$ solve:

$$\max_{\beta_0,\ldots,\beta_p} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

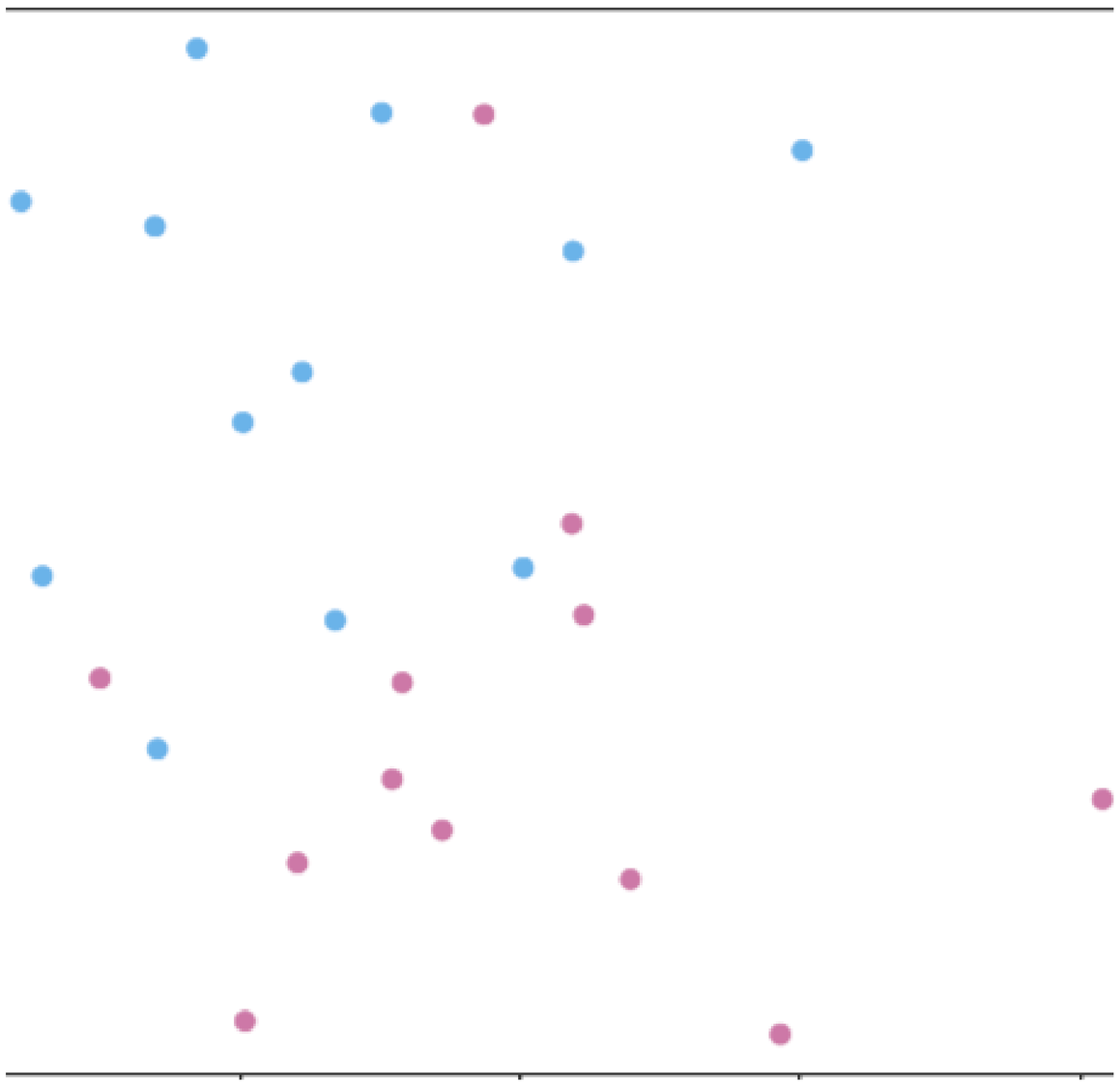
Can be written as a convex optimization problem.

We know how to solve convex optimization problems efficiently to find M and β.

Maximal Margin Classifier

Recall the assumption: Classes can be separated by a linear decision boundary.

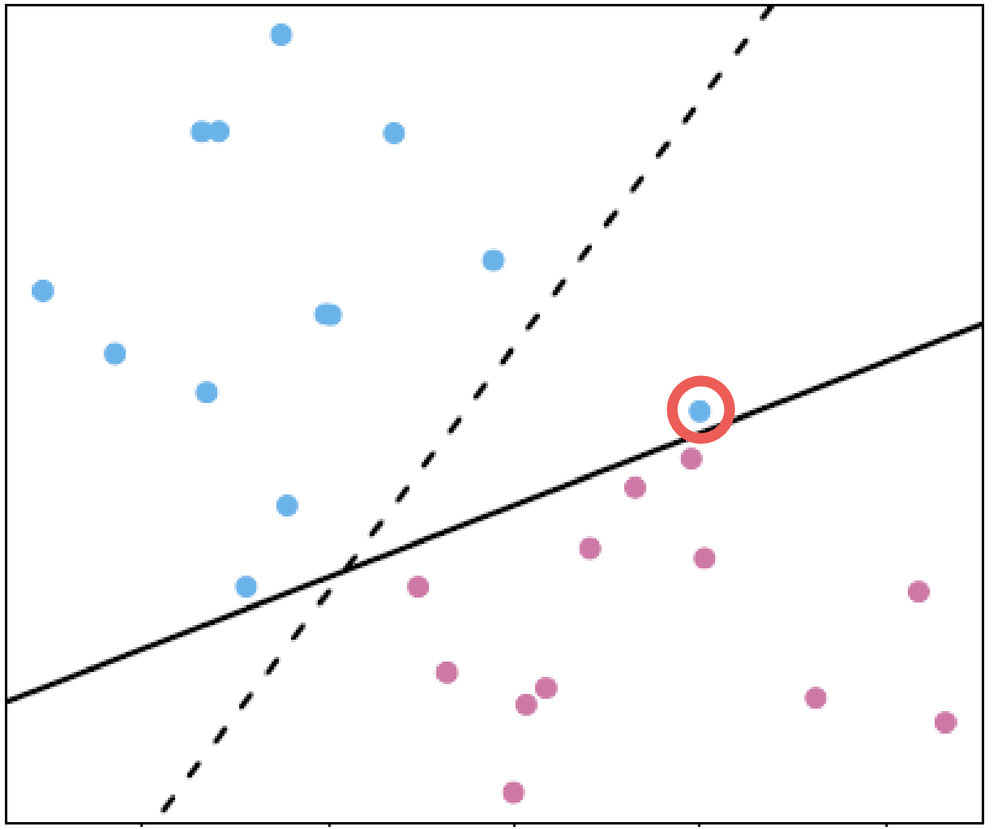
What if there is no separating hyperplane?



Maximal Margin Classifier

Furthermore, notice a disadvantage of the maximal margin classifier:

- Can be sensitive to individual observations
- May overfit training data



Support Vector Classifier

Support Vector Classifier

Like the maximal margin classifier, it looks for a hyperplane to perform classification.

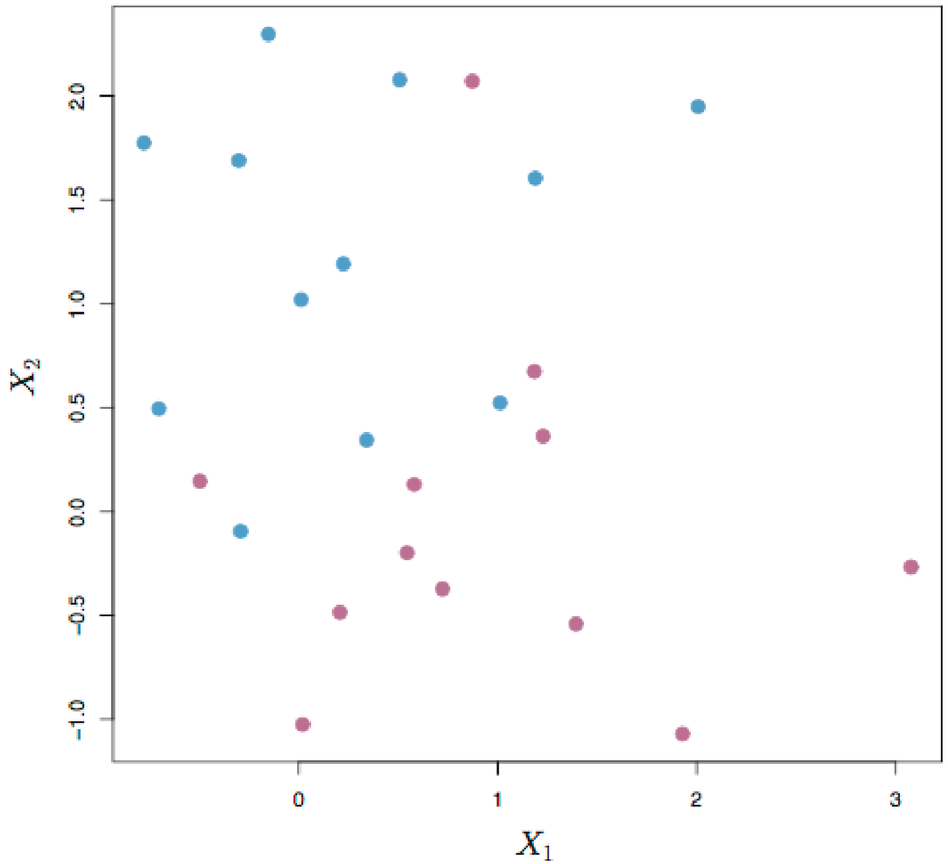


FIGURE 9.4, ISL (8th printing 2017)

Support Vector Classifier

Like the maximal margin classifier, it looks for a hyperplane to perform classification.

However, training samples are allowed to be on the “wrong side” of the margin or hyperplane.

This hyperplane *almost* separates the classes using a “soft margin”.

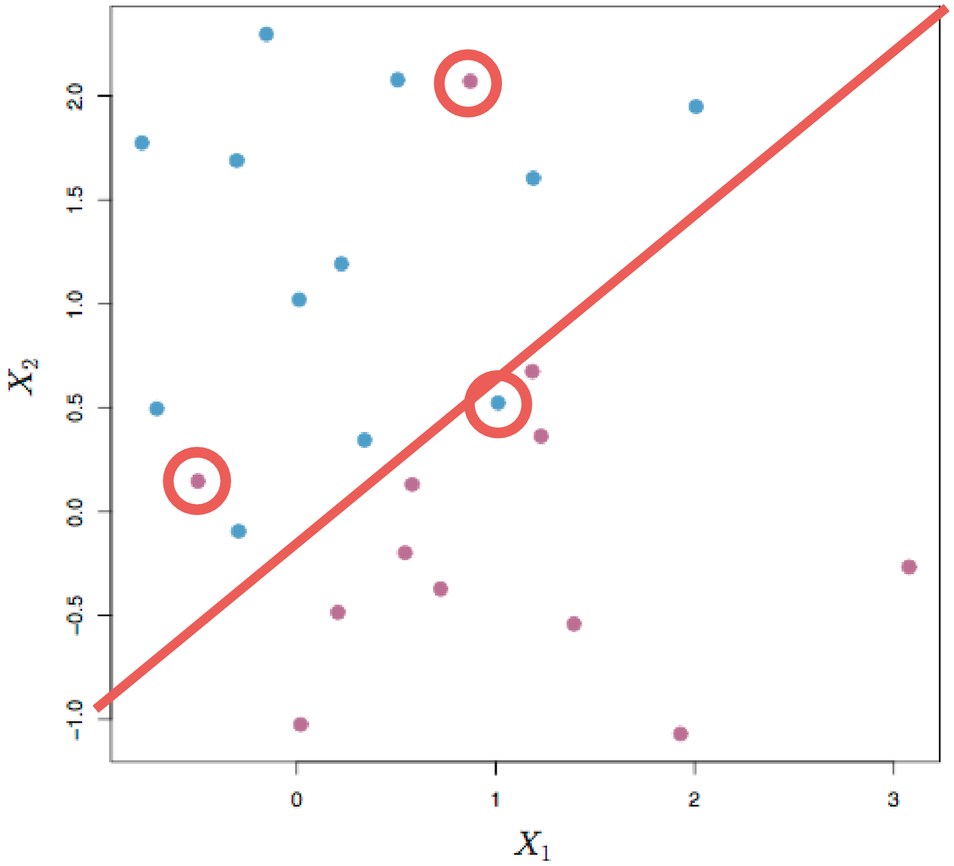


FIGURE 9.4, ISL (8th printing 2017)

Support Vector Classifier

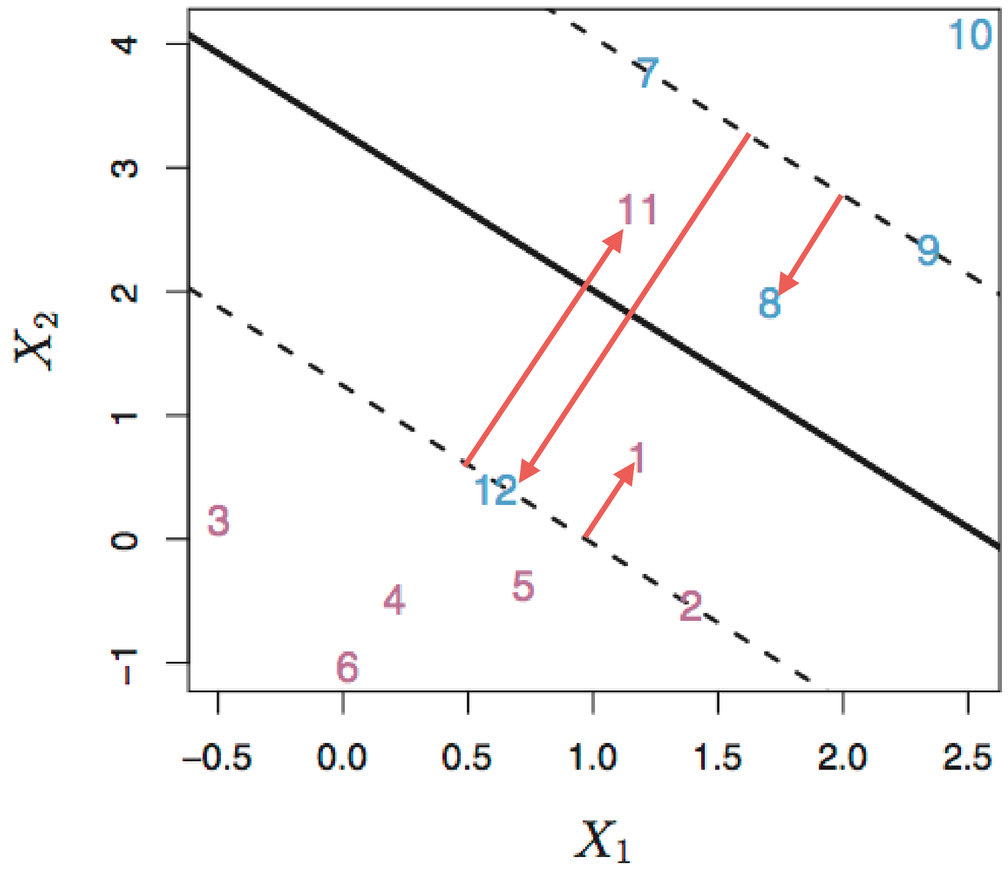


FIGURE 9.6, ISL (8th printing 2017)

Some points are allowed to violate the margin.

Support Vector Classifier

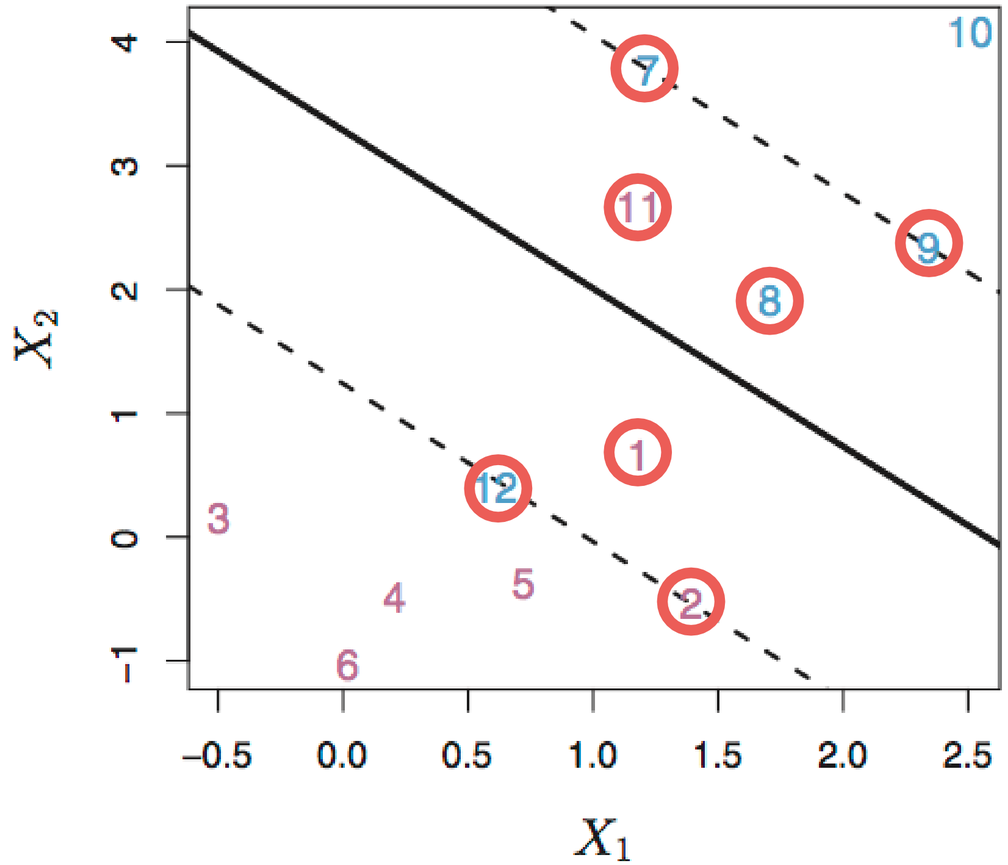


FIGURE 9.6, ISL (8th printing 2017)

Support vector classifiers also have support vectors.

They are points lying directly on the margin, or on the wrong side of the margin for their class.

These observations affect the hyperplane.

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1-\epsilon_i), \quad \forall i$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

maximize the margin, M

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

constraint necessary for well-defined optimization problem

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

training points must be at least distance M from hyperplane, or pay a penalty ε_i

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

training points must be at least distance M from hyperplane, or pay a penalty ε_i

"slack" variable ε_i

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

training points must be at least distance M from hyperplane, or pay a penalty ε_i

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

limit on total penalties. C is a constant.

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

“slack” variable ε_i

Finding Support Vector Classifier

To find the support vector classifier hyperplane, solve:

$$\max_{\beta_0,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n} M$$

$$\text{subject to} \sum_{j=0}^{p} \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \ldots \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \qquad \epsilon_i \geq 0, \quad \forall i$$

In other words, you can violate the margin, but only by a total amount C on your entire dataset.

Support Vector Classifier

- Slack variables ε_i allow for violations of the margin.
  - $\varepsilon_i = 0$: training point is on correct side of margin
  - $\varepsilon_i > 0$: training point violates the margin
  - $\varepsilon_i > 1$: training point is misclassified (wrong side of hyperplane)
- Penalty parameter *C* is the total “budget” for violations.
  - Allows at most *C* misclassifications on training set.

How do we choose C?

As with many things we don't know *a priori* in machine learning, C is a hyperparameter that we tune using cross-validation.

Note that it must be non-negative.

If $C = 0$, we recover the maximal margin classifier (if one exists).

As C goes from small to large, there is a bias-variance tradeoff.

Bias, Variance and C

- Large C
  - Large violation budget
  - Large margin
  - Many support vectors
- Small C
  - Small violation budget
  - Small margin
  - Few support vectors

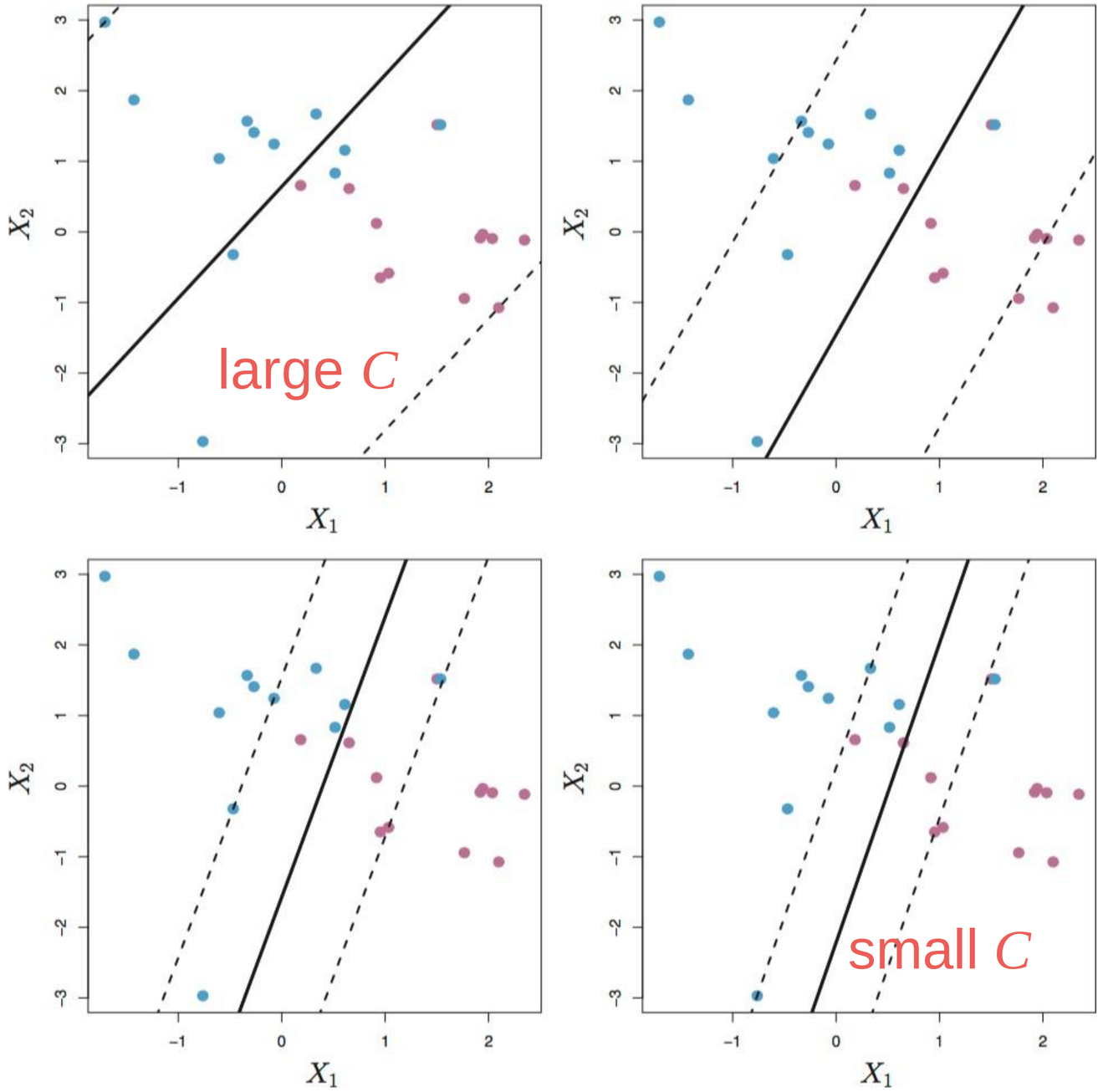


FIGURE 9.7, ISL (8th printing 2017)

Bias, Variance and C

- Large C
  - High bias
  - Low variance
- Small C
  - Low bias
  - High variance

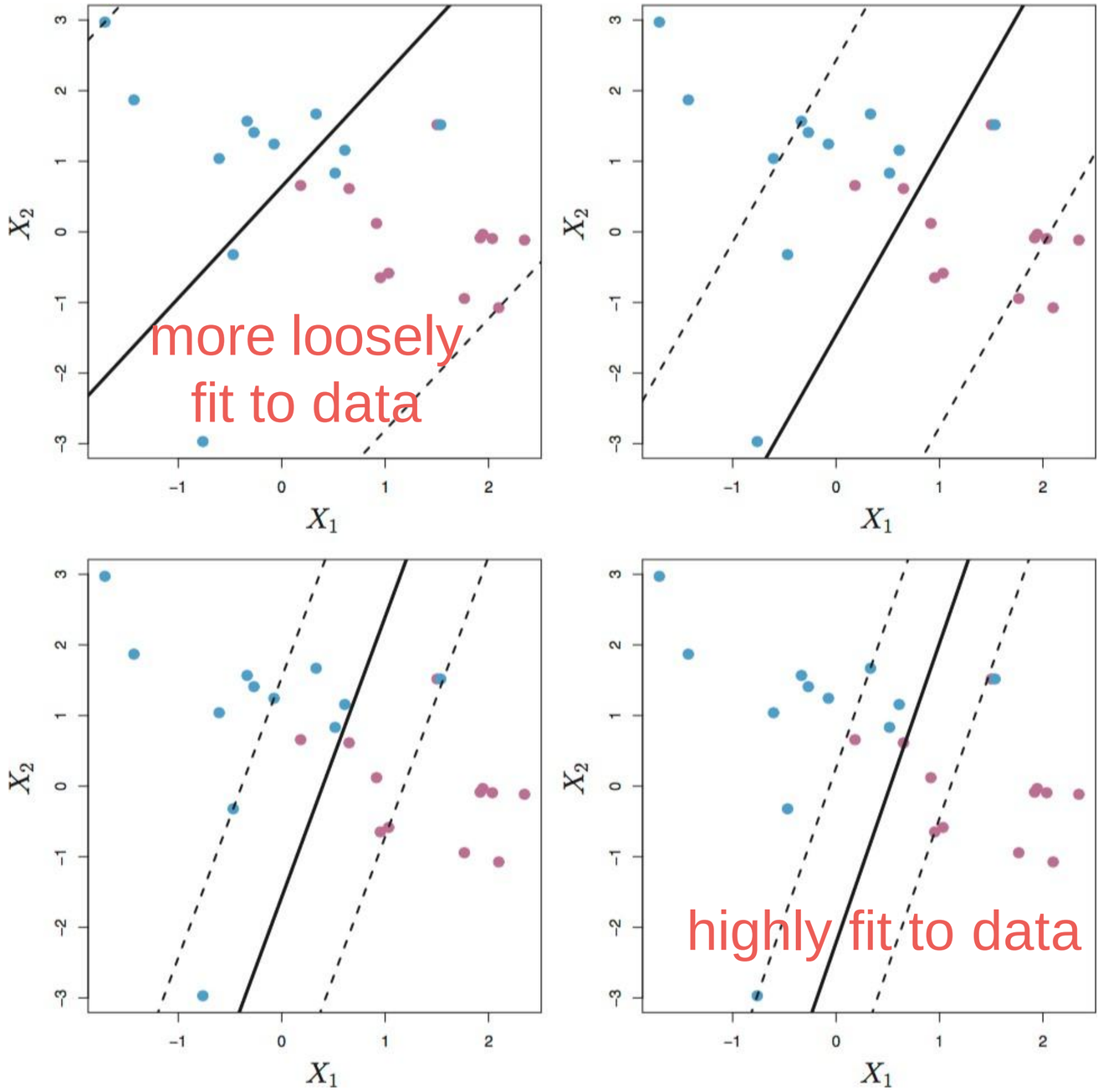


FIGURE 9.7, ISL (8th printing 2017)

Support Vector Classifier

We are still using a *linear* decision boundary.

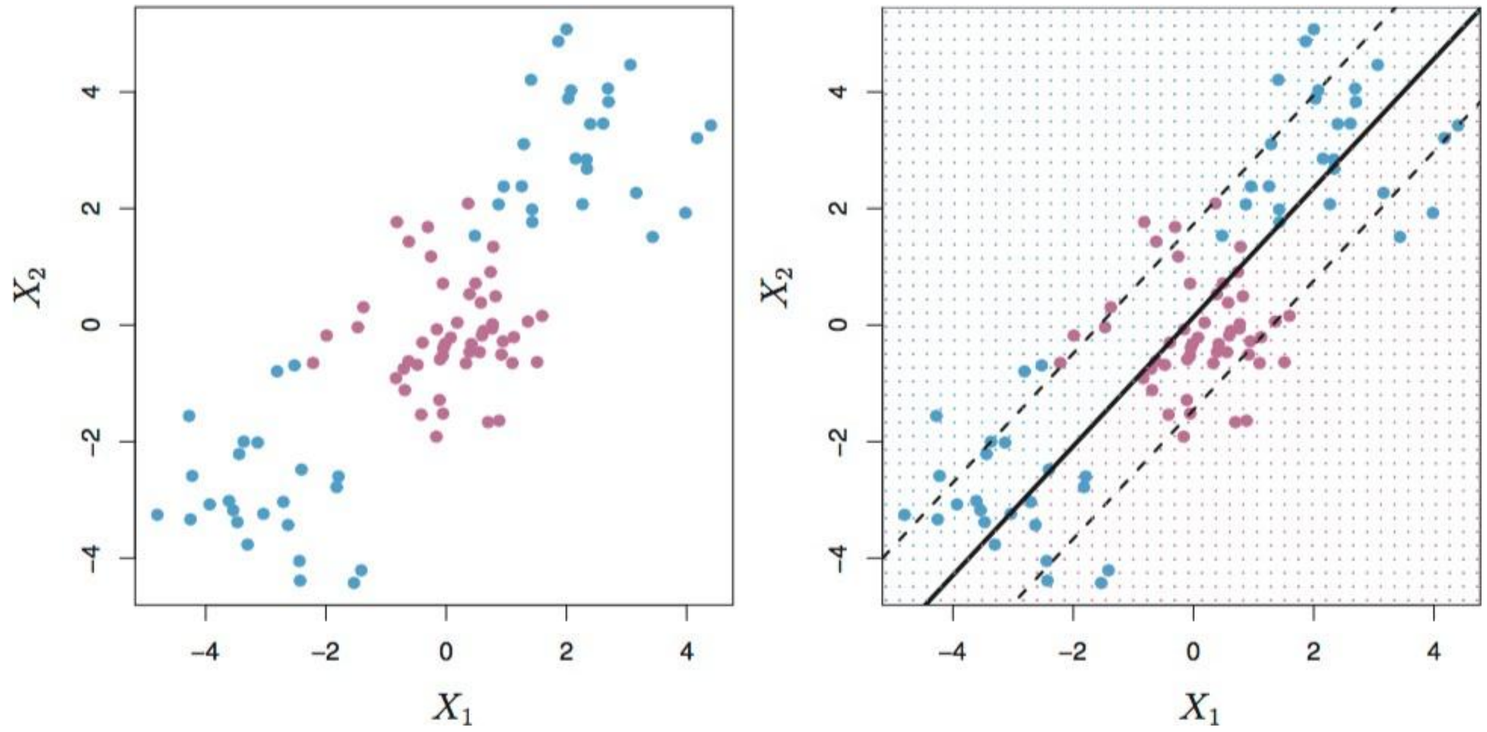
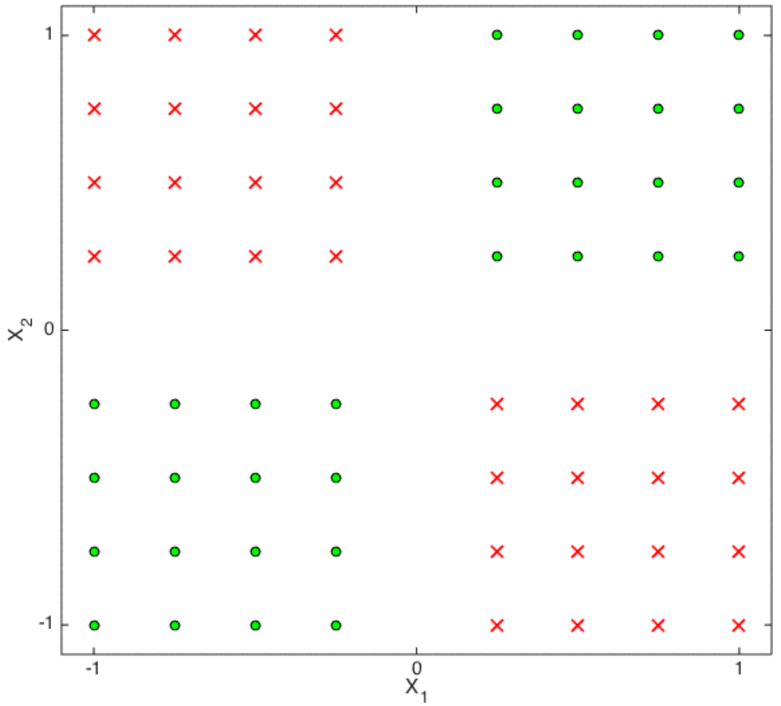


FIGURE 9.8, ISL (8th printing 2017)

Expanding Feature Space

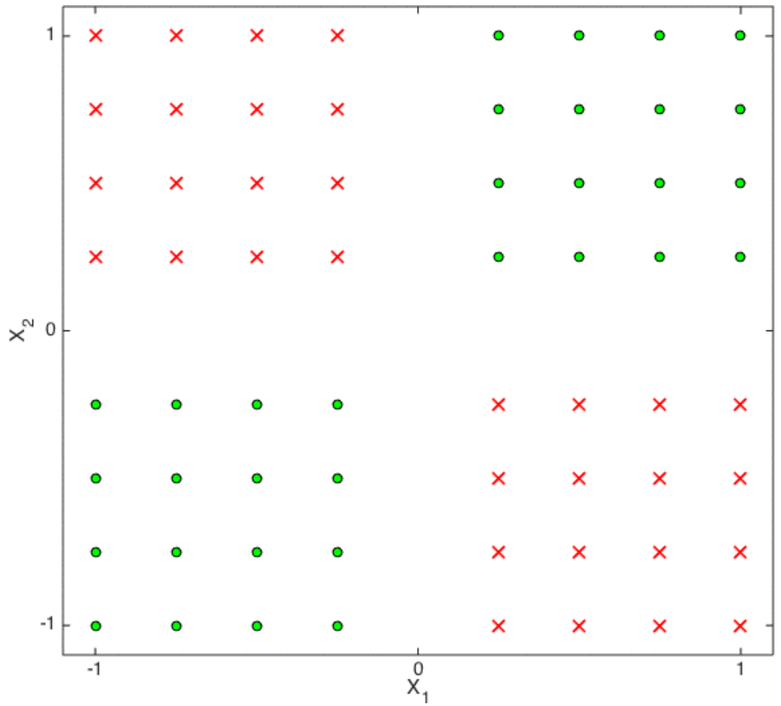
Some datasets are not linearly separable, but they *become* linearly separable when transformed into a *higher* dimensional space.

(Note: Yes, higher dimension also increases chance of overfitting. But in some cases the tradeoff is worthwhile.)



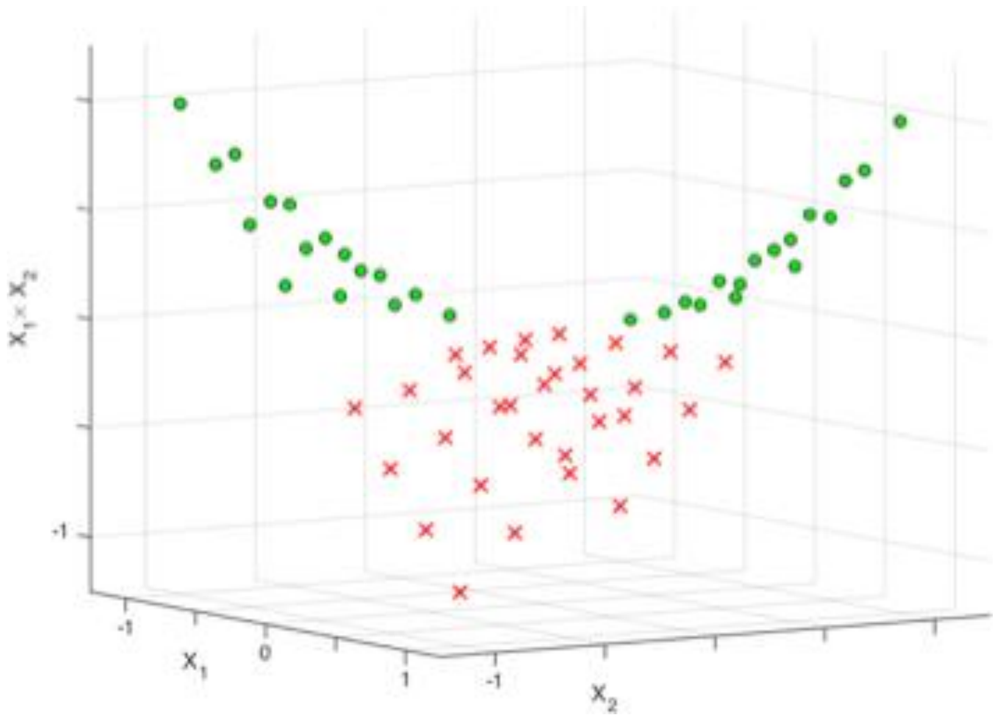
Expanding Feature Space

Original feature space



variables x_1, x_2

New feature space



variables x_1, x_2, x_1x_2

Expanding Feature Space

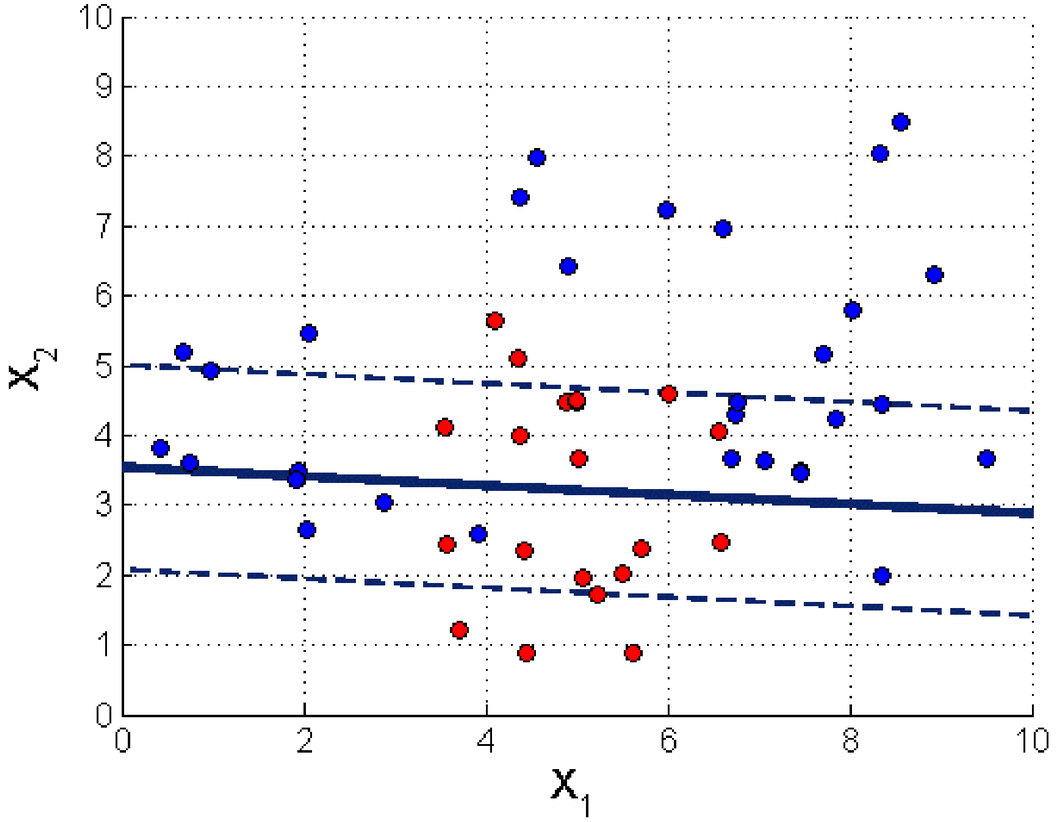
Support vector classifier will find a hyperplane in $2p$ dimensions.

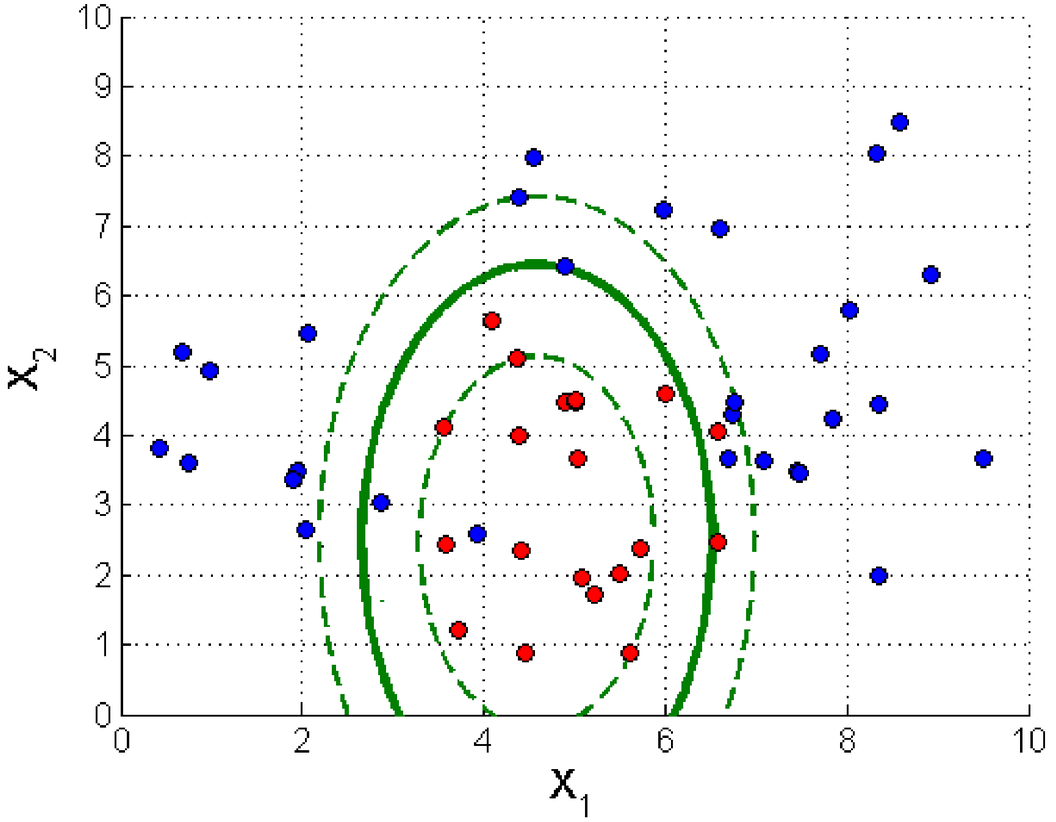
$$\beta_0 + \beta_1 \tilde{X}_1 + \beta_2 \tilde{X}_2 + \ldots + \beta_{2p-1} \tilde{X}_{2p-1} + \beta_{2p} \tilde{X}_{2p} = 0$$

Hyperplane will be non-linear in *original* feature space.
In this case, it is an ellipse.

$$\beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \ldots + \beta_{2p-1} X_p + \beta_{2p} X_p^2 = 0$$

Non-linear Decision Boundary



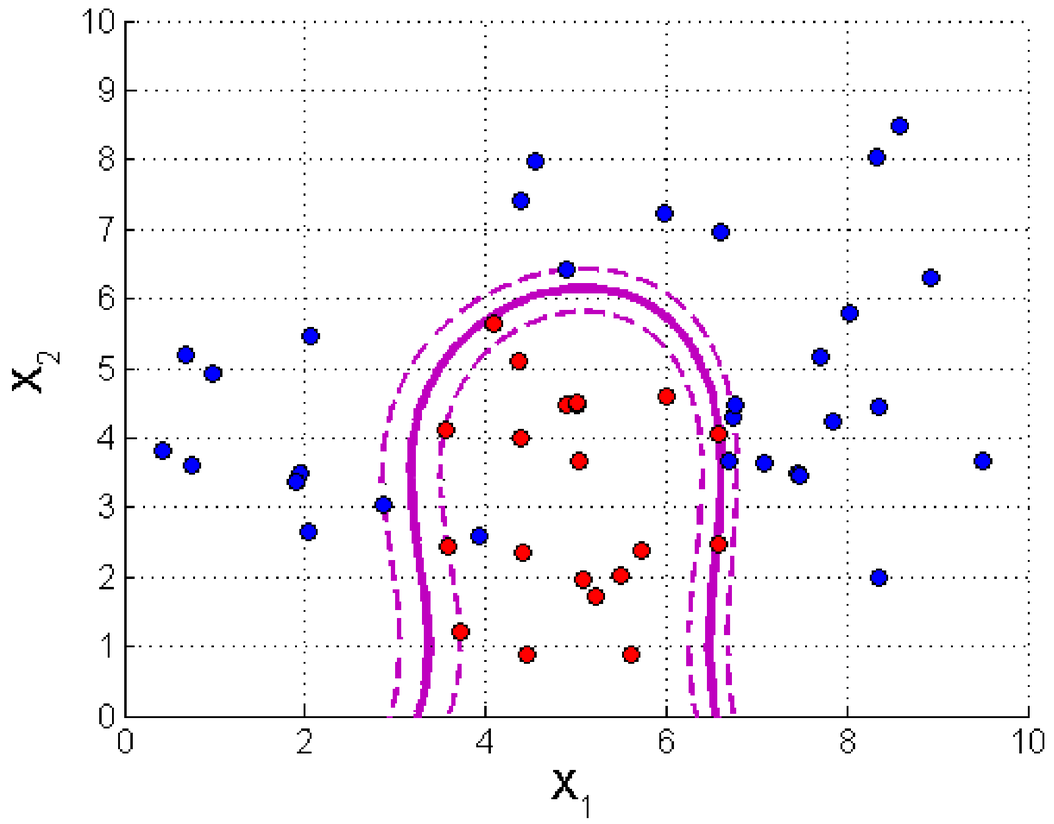


Expanding Feature Space

Can imagine adding higher order polynomial terms, quotients, and more to expand features set.

Large number of features becomes computationally challenging.

We need an efficient way to work with large number of features.



Support Vector Machines

Support Vector Machine (SVM)

Extends the support vector classifier by using **kernel functions** to achieve non-linear decision boundaries.

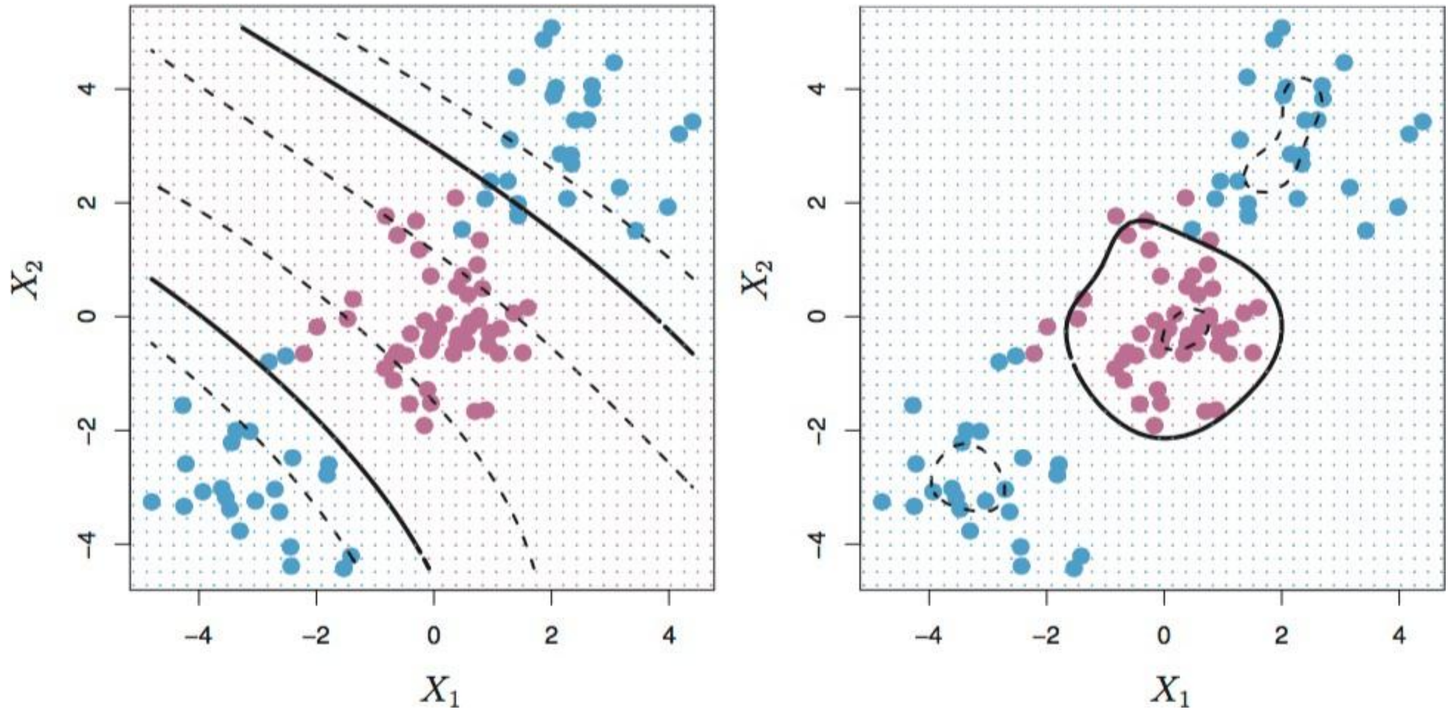


FIGURE 9.9, ISL (8th printing 2017)

Linear SVM

It can be shown that a support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \langle x, x_i \rangle$$

inner product

recall: $f(x) > 0$ is one class
$f(x) < 0$ is another

S is the set of support vectors

where

$$\langle \vec{u}, \vec{v} \rangle = \sum_{j=1}^{p} u_j v_j$$

General SVM

It can be shown that a support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \langle x, x_i \rangle$$

In SVM we replace the inner product with some kernel function

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i K\left(x^{(i)}, x\right)$$

Properties of Kernels

- **Generalization of inner product:**

$$\text{for an explicit feature map} \quad \phi : \mathcal{X} \to \mathcal{X}^{\phi}$$

$$x \mapsto \phi(x)$$

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{X}^{\phi}}$$

- **Symmetric:** $K(x, x') = K(x', x)$
- **Gives a measure of similarity** between X and X'
  - If X and X' are close together, then large
  - If X and X' are far apart, then small

For a more formal definition:
http://mlweb.loria.fr/book/en/constructingkernels.html

Common SVM Kernels

- Linear kernel

$$K(x, x') = \langle x, x' \rangle$$

- Polynomial kernel (degree p)

$$K(x, x') = (1 + \langle x, x' \rangle)^p$$

- Radial basis kernel

$$K(x, x') = \exp\left(-\gamma ||x - x'||^2\right)$$

(an infinite-dimensional feature map!)

Why use kernels?

- Why use kernels instead of explicitly constructing a larger feature space?
- Computational advantage:

$$\phi : \mathbb{R}^p \to \mathbb{R}^P, \quad p \ll P$$

$$K(x, x') = \langle \phi(x), \phi(x') \rangle \quad \text{in } O(p)$$

SVM with Non-Linear Kernels

Cubic polynomial kernel

Radial kernel

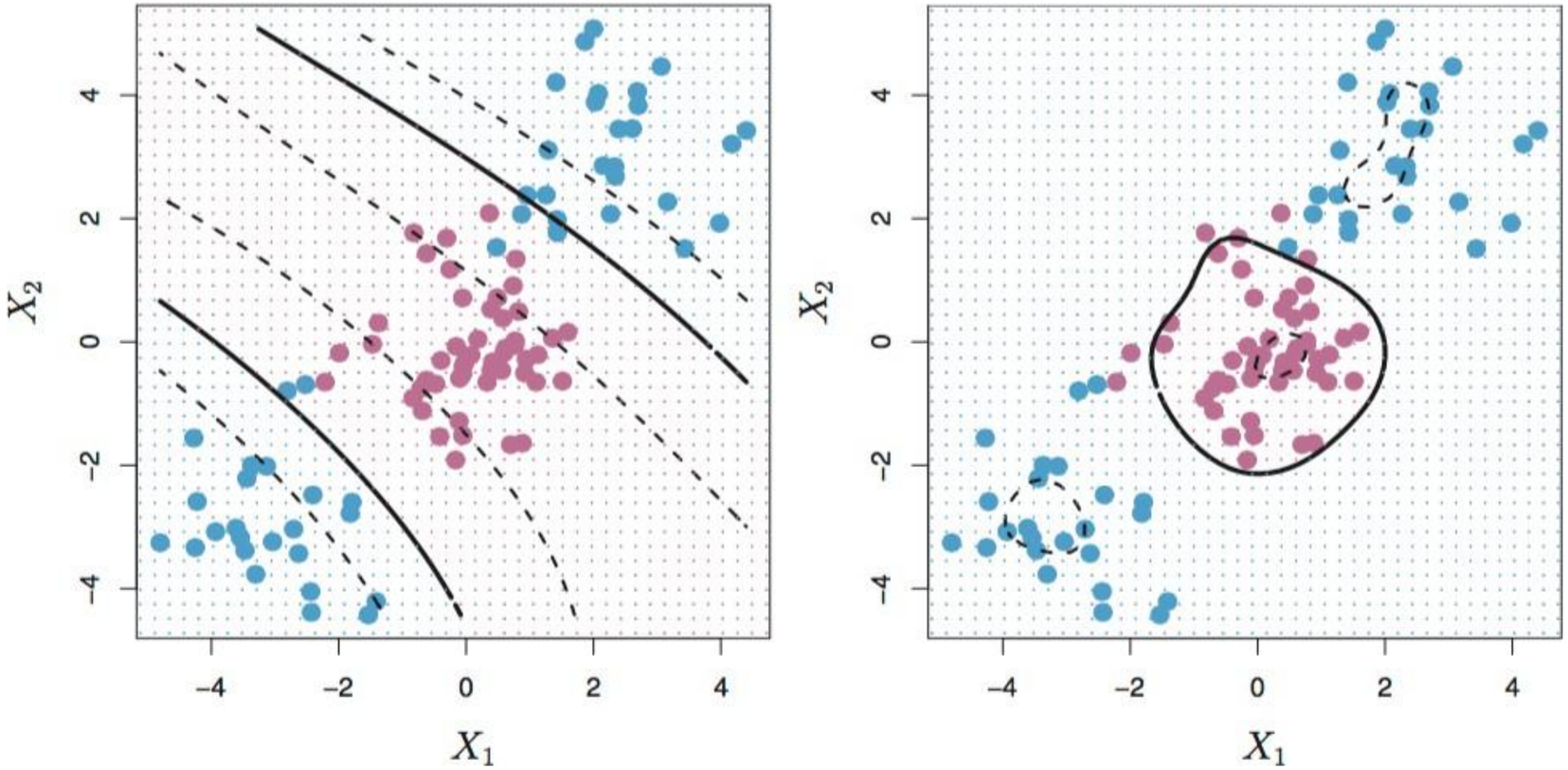


FIGURE 9.9, ISL (8th printing 2017)

SVM Summary

- **Pros:**
  - Regularization parameter *C* helps avoid overfitting
  - Use of kernel gives flexibility in shape of decision boundary
  - Optimization problem is convex — unique solution
- **Cons:**
  - Must tune hyperparameters (e.g. *C*, kernel function)
  - Must formulate as binary classification
  - Difficult to interpret

SVM with 3+ Classes

- SVMs are designed for binary classification, given the nature of a separating hyperplane.
- We can adapt SVMs to perform classification when we have more than 2 classes.
- Popular approaches:
  - One-versus-one
  - One-versus-all

One-versus-one Classification

- Construct an SVM for each pair of classes.
- For k classes, this requires training $k(k\text{-}1)/2$ SVMs.
- To classify a new observation, apply all $k(k\text{-}1)/2$ SVMs to the observation. Take the most frequent class among pairwise results as the predicted class.
- **Con:** computationally expensive for large k.

One-versus-all Classification

- Construct an SVM for each class against the *k*-1 other classes pooled together.
- For *k* classes, this requires training *k* SVMs.
- Distance to separating hyperplane is a proxy for confidence of classification. For new observation, choose the "highest confidence" class as the prediction.
- **Con:** may exacerbate class imbalances, distance to hyperplane may not correspond well to confidence.

References

1. Sherrie Wang, Introduction to Machine Learning, Creig Mellon University, 2019

2. R. Berwick, Village Idiot, MIT University, 2017